- ***1*** Iterates over epochs
- ***2*** Gets the current state of the environment
- ***3*** Predicts the value of the state
- ***4*** Predicts the probability distribution over actions given the state
- ***5*** Samples an action from the policy's action distribution
- ***6*** Predicts the value of the next state
- ***7*** Calculates the advantage as the reward plus the difference between the next state value and the current state value
- ***8*** Reinforces the action that was just taken based on the advantage

This is very simplified pseudocode, but it gets the main idea across. The important part to point out is the advantage calculation. Consider the case where we take an action, we receive reward +10, the value prediction is +5, and the value prediction for the next state is +7. Since future predictions are always less valuable than the currently observed reward, we discount the value of the next state by the gamma discount factor. Our `advantage = 10 + 0.9*7 – 5 = 10 + (6.3 – 5) = 10 + 1.3 = +11.3`. Since the difference between the next state value and the current state value is positive, it increases the overall value of the action we just took, so we will reinforce it more. Notice that the advantage function *bootstraps* because it computes a value for the current state and action based on predictions for a future state.

In this chapter we're going to use our DA2C model on CartPole again, which is episodic, so if we do a full Monte Carlo update where we update after the full episode is complete, `value_next` will always be 0 for the last move since there is no next state when the episode is over. In this case, the advantage term actually reduces to `advantage = reward – value`, which is the value baseline we discussed at the beginning of the chapter. The full advantage expression, $A = r_{t+1} + \gamma * v(s_{t+1}) - v(s_t)$, is used when we do online or *N-step learning*.

